Propuesta de Proyecto: Redacción de Contratos Rurales por Voz

Materia: Lenguajes de Programación
Integrantes: *(completar)*
Docente: *(completar)*
Fecha: 28 de octubre de 2026
Estado: borrador inicial. Los campos de datos y las plantillas se irán ajustando durante el desarrollo.

---

1. Resumen

Se propone un sistema que **redacta contratos de arrendamiento rural a partir de lo que una persona dice por voz**. El usuario describe el acuerdo hablando (por ejemplo, *"arriendo un lote de 5 hectáreas para 20 vacas de raza Holstein para ganadería de leche por 12 meses"*). El sistema reconoce las palabras clave, valida que la frase tenga una estructura correcta y rellena automáticamente una **plantilla de contrato** según el tipo de arrendamiento.

El proyecto aplica de forma directa los temas de la materia: **expresiones regulares, autómatas finitos, gramáticas libres de contexto, autómatas a pila y máquinas de Turing**.

2. Problema y público objetivo

En el sector campesino, muchos acuerdos de arrendamiento de tierra se hacen de palabra o con documentos informales. Redactar un contrato formal requiere escribir mucho y conocer términos legales. Hablar es más natural y accesible que escribir, sobre todo para personas con poca experiencia digital.

Objetivo: que una persona pueda decir las condiciones del arriendo y obtener un borrador de contrato listo para revisar.

3. Alcance

Incluye:

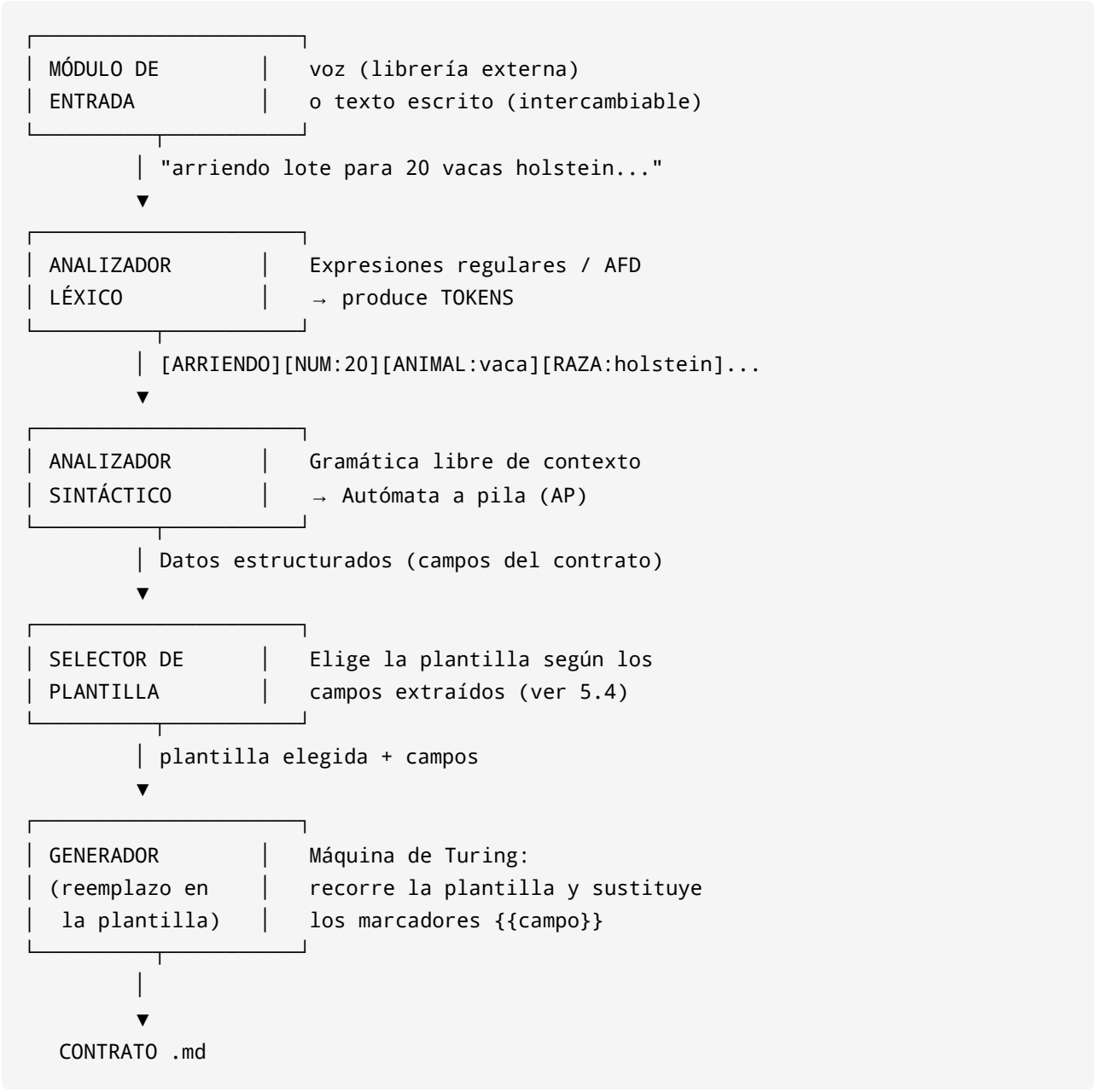
- Entrada por voz convertida a texto (reconocimiento de voz de una librería existente; no se desarrolla el reconocedor) o, de forma intercambiable, texto escrito.
- Un conjunto reducido de tipos de arrendamiento, cada uno con su plantilla (ver sección 6), y selección automática de la plantilla que mejor se ajusta a lo dictado (ver 5.4).
- Extracción de datos: partes, lote, cantidad de animales, tipo de animal, raza, uso, duración, valor.

- Generación del contrato en texto (.md, con posibilidad de exportar luego).

No incluye (por ahora):

- Validez jurídica del contrato: el resultado es un **borrador** que debe ser revisado por un profesional.
- Reconocimiento de voz propio, ni todos los acentos o regionalismos.
- Firma digital.

4. Arquitectura general



4.1 Módulo de entrada intercambiable

El núcleo teórico del proyecto empieza **cuando ya existe el texto**. Por eso la entrada es un módulo independiente:

```
[ Entrada ]  →  TEXTO  →  Léxico → AP → Selector → MT → contrato
    │
    ├─ micrófono + librería de voz a texto
    ├─ dictado del teclado del celular
    └─ cuadro de texto escrito (pruebas y plan B)
```

Se desarrolla y prueba todo con texto escrito, y la voz se conecta al final. Si el reconocimiento de voz falla en la demostración, el sistema sigue funcionando con texto.

5. Fundamento teórico: qué modelo se usa en cada etapa

Etapa	Modelo	Por qué
Léxico (reconocer números, animales, razas)	**Expresiones regulares → AFN/AFD** (construcción de Thompson)	Los tokens son lenguajes regulares
Sintáctico (validar la estructura de la frase)	**Gramática libre de contexto → Autómata a pila**	Hay estructuras anidadas (ver 5.2) que un autómata finito no puede manejar
Generación (rellenar la plantilla)	**Máquina de Turing**	La cinta contiene la plantilla; la máquina la recorre, detecta marcadores y los reemplaza, lo que implica leer, escribir y mover la cabeza en ambos sentidos

Por qué no basta con autómatas finitos. Los modelos forman una jerarquía y cada nivel reconoce más que el anterior:

```
AFD / AFN  (lenguajes regulares)
  ⊂ Autómata a pila  (lenguajes libres de contexto)
    ⊂ Máquina de Turing  (lenguajes recursivamente enumerables)
```

Un AFD solo tiene memoria finita (sus estados), por lo que no puede contar niveles de anidación. El autómata a pila agrega una pila, y la máquina de Turing agrega una cinta que puede leer, escribir y recorrer en ambos sentidos. El AFD se usa únicamente en el analizador léxico; el centro del proyecto es el **AP** y la **MT**.

5.1 Etapa léxica con expresiones regulares

Se usa λ para la cadena vacía. Definiciones:

```
DIGITO = 0|1|2|3|4|5|6|7|8|9
NUM    = DIGITO DIGITO*
```

```
ANIMAL = (vaca|toro|caballo|cabra|oveja|cerdo) (s|λ)
```

Paso a paso con `ANIMAL`, cadena `vacas`:

```
1. Se lee "vaca"   → coincide con la alternativa vaca
2. Queda "s"       → se evalúa (s|λ): se toma la rama s
3. Cadena consumida completa → ACEPTA  → token ANIMAL("vaca")
```

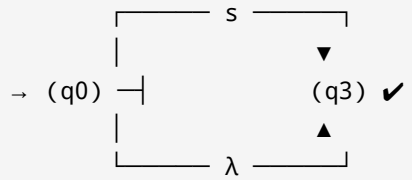
Paso a paso con `vaca` (singular):

```
1. Se lee "vaca"   → coincide con la alternativa vaca
2. No queda nada   → se evalúa (s|λ): se toma la rama λ
3. Cadena consumida completa → ACEPTA
```

Paso a paso con `NUM`, cadena `20`:

```
1. '2' → coincide con DIGITO (primer DIGITO obligatorio)
2. '0' → coincide con DIGITO*  (una repetición)
3. Fin de cadena → ACEPTA → token NUM(20)
```

Construcción de Thompson para `s|λ` (diagrama del AFN):



> En el documento final se incluirá el AFN completo de `NUM` y `ANIMAL`, y su conversión a AFD.

5.2 Etapa sintáctica con gramática y autómata a pila

Gramática simplificada de la lista de animales (el símbolo `λ` es la producción vacía):

```
LISTA   → ITEM RESTO
RESTO   → , ITEM RESTO | y ITEM | λ
ITEM    → NUM ANIMAL RAZAOPT SUBOPT
RAZAOPT → de raza RAZA | λ
SUBOPT  → de los cuales LISTA fin | λ
```

La parte `SUBOPT` permite frases **anidadas**, por ejemplo:

> *"20 vacas de raza holstein **de los cuales** 5 terneras **fin** y 3 caballos"*

Para saber cuándo cierra cada grupo hace falta recordar el nivel de anidación: **eso lo resuelve la pila**.

Árbol de derivación (AST) de `20 vacas y 3 caballos`:

```
          LISTA
         /     \
      ITEM     RESTO
     /  |  \     /  |  \
  NUM ANIMAL  y  ITEM
  20  vacas      / | \
              NUM ANIMAL
               3  caballos
```

Comportamiento de la pila con `... de los cuales 5 terneras fin ...`:

```
Evento                       Pila (tope a la derecha)
_____________________________   _____________________________

inicio                       Z0
lee "de los cuales"          Z0 SUB        ← apila
lee "5 terneras"             Z0 SUB        (procesa ITEM)
lee "fin"                    Z0            ← desapila
fin de entrada, pila = Z0    ACEPTA
```

5.3 Etapa generadora con máquina de Turing

La cinta contiene la plantilla con marcadores:

```
 ... | E | l |   | a | r | r | e | n | d | a | t | a | r | i | o |   | {{ | n | o | m
| b | r | e | }} | ...
                                                                    ↑ cabeza
```

Funcionamiento resumido:

```
Estado q_buscar : avanza a la derecha hasta encontrar "{{"
Estado q_leer   : lee el nombre del campo hasta "}}"
Estado q_borrar : sustituye el marcador por blancos
Estado q_escribir: escribe el valor del campo (viene de la etapa sintáctica)
Estado q_buscar : continúa hacia la derecha
Estado q_fin    : no quedan marcadores → ACEPTA
```

5.4 Selección automática de la plantilla

Se cargan todas las plantillas disponibles y el sistema elige la que mejor se adapta a lo dictado. Cada plantilla declara los campos que necesita; se cuenta cuántos de esos campos aparecieron en la frase:

```
Campos extraídos: lote, hectáreas, animal, cantidad, raza, uso

Plantilla PASTOREO : necesita lote, hectáreas, animal, cantidad, raza, uso   → 6 de
```

```
6
 Plantilla CULTIVO  : necesita lote, hectáreas, cultivo, insumos              → 2 de
4
 Plantilla MIXTA    : necesita ambos conjuntos                                → 6 de
9

 Se elige PASTOREO (mayor coincidencia)
```

Reglas:

- Si hay empate o la coincidencia es baja, el sistema pregunta por los datos que faltan.
- Los campos que faltan quedan marcados como `{{campo}}` pendientes en el borrador.
- Esta comparación se implementa sobre la cinta de la máquina de Turing: la cinta contiene los campos extraídos y los marcadores de cada plantilla, y la máquina los recorre y compara, de modo que la selección también usa el modelo central del proyecto y no solo un autómata finito.

5.5 Flexibilidad en el orden de las cláusulas

No se obliga al usuario a seguir un orden ni a empezar con la palabra "arriendo". Las frases siguientes son válidas y producen el mismo contrato:

```
"arriendo el lote 20 para 20 toros gir por 30 días"
"los 20 toros gir se arriendan en el lote 20 por 30 días"
"por 30 días, lote 20, 20 toros gir"
```

Para lograrlo, la gramática se define como una **secuencia de cláusulas independientes**, cada una con su propio marcador de inicio:

```
CONTRATO → CLAUSULA CONTRATO | λ
CLAUSULA → ANIMALES | LOTE | VALOR | FECHA | DURACION | USO | VERBO
LOTE     → en el lote ID | lote ID
VALOR    → por un valor de MONTO | valor MONTO
FECHA    → desde el DIA de MES (de ANIO | λ) | el DIA de MES (de ANIO | λ)
DURACION → por NUM UNIDAD
USO      → para (ganadería|pastoreo|cultivo|producción de ...)
VERBO    → arriendo | se arriendan | se arrienda | λ
```

Cómo se evita la ambigüedad: el autómata mira el token siguiente (símbolo de anticipación):

```
"por" + "un valor de"  → VALOR
"por" + NUM + UNIDAD   → DURACION
"en el lote" + NUM     → LOTE (el NUM es el número de lote)
"los" + NUM + ANIMAL   → ANIMALES (el NUM es la cantidad)
```

Así, el `20` de "los 20 toros" y el `20` de "lote 20" no se confunden.

ER paso a paso con λ, para la fecha (`15 de noviembre de 2026`):

```
FECHA = NUM de MES (de NUM | λ)

1. "15"        → NUM = DIGITO DIGITO*          → NUM(15)
2. "de"        → texto fijo
3. "noviembre" → alternativa de MES            → MES(11)
4. "de 2026"   → se toma la rama (de NUM) de (de NUM | λ)
5. Fin         → ACEPTA → FECHA(15, 11, 2026)
```

Si el usuario dice solo `15 de noviembre`, en el paso 4 se toma la rama **λ** y el año queda pendiente.

5.6 Guía inicial para el usuario

Como no se impone un orden, al comenzar el sistema muestra (o dice en voz alta) una guía con la información que debe mencionar:

```
┌──────────────────────────────────────────────────────┐
│  Para redactar su contrato, mencione:                │
│                                                      │
│   ✔ Tipo de animal (y raza, si aplica)               │
│   ✔ Número de lote                                   │
│   ✔ Fecha                                            │
│   ✔ Valor del arrendamiento                          │
│   ✔ Para qué se usará (uso)                          │
│                                                      │
│  Opcional: cantidad de animales, duración            │
│  Puede decirlo en el orden que prefiera.             │
└──────────────────────────────────────────────────────┘
```

Después de que el usuario habla, el sistema compara lo recibido con la lista y pregunta solo por lo que falta:

```
Campos requeridos : animal, lote, fecha, valor, uso
Campos detectados : animal ✔  lote ✔  valor ✔  fecha ✘  uso ✘

Sistema: "Falta la fecha y para qué se usará el lote."
Usuario: "desde el 15 de noviembre para ganadería de leche"
Campos detectados : animal ✔  lote ✔  valor ✔  fecha ✔  uso ✔  → COMPLETO
```

Cuando todos los campos requeridos están completos, se pasa a la selección de la plantilla (5.4) y a la generación del contrato.

Campos configurables (borrador). La lista anterior es una primera versión. Cada plantilla declara sus propios campos obligatorios y opcionales, y el sistema los lee de esa declaración en lugar de tenerlos fijos en el código:

```
PLANTILLA PASTOREO
  obligatorios : animal, lote, fecha, valor, uso
  opcionales   : raza, cantidad, duración

PLANTILLA CULTIVO
  obligatorios : lote, fecha, valor, cultivo
  opcionales   : insumos, duración
```

- Un campo opcional que no se dice no genera preguntas, y la cláusula que lo menciona se omite del contrato: la máquina de Turing borra esa parte de la plantilla.
- Si se define que un dato es obligatorio por requisitos legales, basta con moverlo de opcional a obligatorio, sin modificar el autómata ni la gramática.
- Los campos definitivos se decidirán con pruebas y, para la parte legal, con asesoría de un profesional.

6. Tipos de arrendamiento y plantillas

Plantilla	Campos clave
Arriendo de lote para pastoreo	lote, hectáreas, tipo y cantidad de animales, raza, duración, valor
Arriendo de lote para cultivo	lote, hectáreas, cultivo, duración, valor, quién aporta insumos
Arriendo mixto (pastoreo + cultivo)	combinación de los anteriores

Cada plantilla tendrá **cláusulas fijas** (obligaciones, terminación, resolución de conflictos) y **marcadores variables** como `{{cantidad_animales}}`, `{{tipo_animal}}`, `{{raza}}`, `{{uso}}`.

Ejemplo de entrada y salida:

```
ENTRADA (voz → texto):
"arriendo lote La Esperanza de 5 hectáreas para 20 vacas de raza
 holstein para producción de leche por 12 meses"

TOKENS:
[ARRIENDO][LOTE:La Esperanza][NUM:5][UNIDAD:hectáreas][NUM:20]
[ANIMAL:vaca][RAZA:holstein][USO:leche][NUM:12][UNIDAD:meses]

SALIDA (fragmento del contrato):
"CLÁUSULA SEGUNDA. El ARRENDATARIO podrá mantener en el lote La Esperanza
 (5 hectáreas) un máximo de 20 vacas de raza Holstein, destinadas
 exclusivamente a la producción de leche, durante 12 meses."
```

7. Tecnologías propuestas

- Lenguaje: *(Python / Java / otro, a definir con el docente)*
- Voz a texto: librería existente (ej. SpeechRecognition o la API del navegador)
- Implementación propia (sin generadores automáticos) del léxico, el AP y la MT, para que el proyecto muestre la teoría de la materia

8. Plan de trabajo

Fase	Actividad
1	Definir tokens, ER y gramática. Diseñar autómatas en papel
2	Implementar analizador léxico (ER → AFD)
3	Implementar analizador sintáctico (AP) y extracción de campos
4	Implementar la máquina de Turing de reemplazo y las plantillas
5	Integrar la voz, pruebas con frases reales y documentación

9. Riesgos

- El reconocimiento de voz puede equivocarse con nombres propios y regionalismos. *Mitigación:* mostrar el texto reconocido y permitir corregirlo antes de generar.
- La variedad de formas de hablar es enorme. *Mitigación:* limitar el vocabulario y las estructuras de frase aceptadas, y documentarlas.
- Alcance excesivo. *Mitigación:* empezar con una sola plantilla (pastoreo) y ampliar si hay tiempo.

10. Entregables

1. Código fuente del sistema.
2. Documento con los autómatas, la gramática y las ER diseñadas.
3. Demostración con al menos 3 contratos generados por voz.

11. Preguntas para el docente

1. ¿Se considera suficiente combinar ER/AFD, AP y MT en el proyecto, o prefiere que nos centremos en un solo modelo?
2. ¿Podemos usar una librería externa de voz a texto?
3. ¿Qué lenguaje de implementación recomienda o exige?
4. ¿El alcance (una plantilla base ampliable) le parece adecuado?

Este proyecto genera borradores informativos y no reemplaza la asesoría de un abogado.